

# JAVA OOP EXPERIMENTS

## Practical Record — 6 Separate Programs

Experiments: Abstraction, Classes & Objects, Constructors & Encapsulation, Inheritance, Method Overloading, Method Overriding

## Common System Requirements

**Hardware:** Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

**Software:** Windows/Linux/macOS, JDK 17/JDK 21, VS Code/Eclipse/IntelliJ IDEA/NetBeans.

# EXPERIMENT 01 — IMPLEMENTATION OF ABSTRACTION USING ABSTRACT CLASS

|               |                                                    |
|---------------|----------------------------------------------------|
| Experiment No | 01                                                 |
| Date          | 17/09/2026                                         |
| Title         | Implementation of Abstraction using Abstract Class |

## ■ Objective / Aim

To write a Java program to demonstrate abstraction using an abstract class and abstract method.

## ■ System Requirements

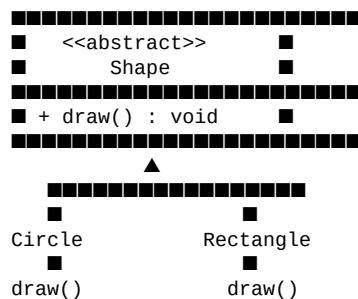
**Hardware:** Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

**Software:** Windows/Linux/macOS, JDK 17/JDK 21, Java IDE/Text Editor.

## ■ Algorithm / Logic Description

1. Create an abstract class named Shape.
2. Declare an abstract method draw().
3. Create a Circle class that extends Shape.
4. Implement draw() in Circle.
5. Create a Rectangle class that extends Shape.
6. Implement draw() in Rectangle.
7. Create Shape references for Circle and Rectangle.
8. Call the draw() methods and display the output.

## ■ UML Class Diagram



## ■ Source Code

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

public class Abstraction {
    public static void main(String[] args) {
        Shape s1 = new Circle();
    }
}
```

```
        Shape s2 = new Rectangle();

        s1.draw();
        s2.draw();
    }
}
```

## ■ Compilation & Execution Commands

```
Save as: Abstraction.java
javac Abstraction.java
java Abstraction.java
```

## ■■ Output Screen

```
Drawing Circle
Drawing Rectangle
```

## ■ Result

Thus, the Java program demonstrating abstraction using an abstract class was successfully implemented and executed.

# EXPERIMENT 02 — IMPLEMENTATION OF CLASSES AND OBJECTS IN JAVA

|               |                                               |
|---------------|-----------------------------------------------|
| Experiment No | 02                                            |
| Date          | 17/09/2026                                    |
| Title         | Implementation of Classes and Objects in Java |

## ■ Objective / Aim

To write a Java program to demonstrate the concept of classes and objects by creating a class containing book details and displaying them using an object.

## ■ System Requirements

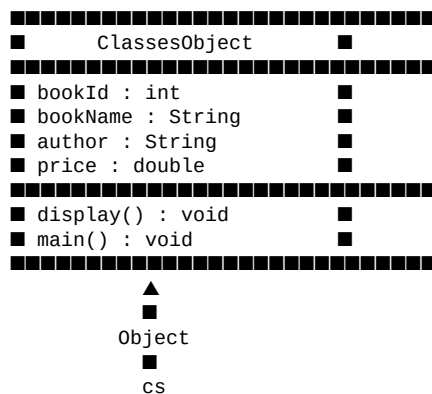
**Hardware:** Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

**Software:** Windows/Linux/macOS, JDK 17/JDK 21, Java IDE/Text Editor.

## ■ Algorithm / Logic Description

1. Create a class named ClassesObject.
2. Declare bookId, bookName, author, and price.
3. Create a display() method.
4. Create an object cs of ClassesObject.
5. Assign values to the object variables.
6. Call the display() method.
7. Display the book details.

## ■ UML Class Diagram



## ■ Source Code

```
public class ClassesObject {

    int bookId;
    String bookName;
    String author;
    double price;

    void display() {
        System.out.println("Book ID: " + bookId);
        System.out.println("Book Name: " + bookName);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
    }

    public static void main(String[] args) {

        ClassesObject cs = new ClassesObject();

        cs.bookId = 18;
```

```
        cs.bookName = "Saipublishers";  
        cs.author = "sai";  
        cs.price = 1000;  
  
        cs.display();  
    }  
}
```

## ■ Compilation & Execution Commands

```
Save as: ClassesObject.java  
javac ClassesObject.java  
java ClassesObject.java
```

## ■■ Output Screen

```
Book ID: 18  
Book Name: Saipublishers  
Author: sai  
Price: 1000.0
```

## ■ Result

Thus, the Java program demonstrating classes and objects was successfully implemented and executed.

## EXPERIMENT 03 — IMPLEMENTATION OF CONSTRUCTORS AND ENCAPSULATION

|               |                                                  |
|---------------|--------------------------------------------------|
| Experiment No | 03                                               |
| Date          | 17/09/2026                                       |
| Title         | Implementation of Constructors and Encapsulation |

### ■ Objective / Aim

To write a Java program to demonstrate constructors and encapsulation using private data members, constructors, getters, and setters.

### ■ System Requirements

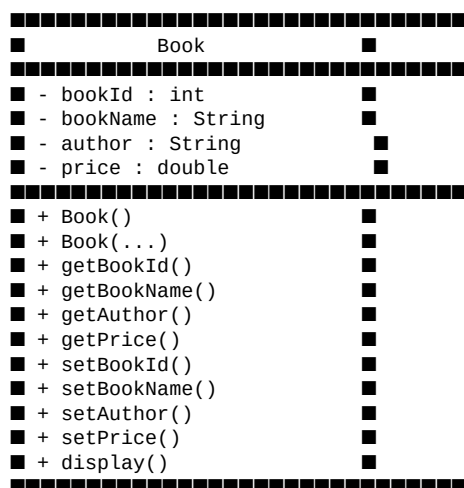
**Hardware:** Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

**Software:** Windows/Linux/macOS, JDK 17/JDK 21, Java IDE/Text Editor.

### ■ Algorithm / Logic Description

1. Create a class named Book.
2. Declare private variables for book details.
3. Create a default constructor.
4. Create a parameterized constructor.
5. Create getter methods to retrieve values.
6. Create setter methods to modify values.
7. Create a display() method.
8. Create objects using both constructors.
9. Modify values using setters and access a value using a getter.
10. Display the results.

### ■ UML Class Diagram



### ■ Source Code

```
class Book {

    private int bookId;
    private String bookName;
    private String author;
    private double price;

    // Default constructor
    Book() {
```

```

        bookId = 0;
        bookName = "Unknown";
        author = "Unknown";
        price = 0;
    }

    // Parameterized constructor
    Book(int bookId, String bookName, String author, double price) {
        this.bookId = bookId;
        this.bookName = bookName;
        this.author = author;
        this.price = price;
    }

    // Getters
    int getBookId() {
        return bookId;
    }

    String getBookName() {
        return bookName;
    }

    String getAuthor() {
        return author;
    }

    double getPrice() {
        return price;
    }

    // Setters
    void setBookId(int bookId) {
        this.bookId = bookId;
    }

    void setBookName(String bookName) {
        this.bookName = bookName;
    }

    void setAuthor(String author) {
        this.author = author;
    }

    void setPrice(double price) {
        this.price = price;
    }

    void display() {
        System.out.println("Book ID: " + bookId);
        System.out.println("Book Name: " + bookName);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
    }
}

public class ConstructionsAndEncasoulation {

    public static void main(String[] args) {

        Book b1 = new Book();

        // Parameterized constructor
        Book b2 = new Book(101, "Java", "James Gosling", 500);

        System.out.println("Book 1:");
        b1.display();

        System.out.println("\nBook 2:");
        b2.display();

        // Setter
        b1.setBookId(102);
        b1.setBookName("C Programming");
        b1.setAuthor("Dennis Ritchie");
        b1.setPrice(400);

        System.out.println("\nAfter using Setter:");
        b1.display();
        // Getter

```

```
        System.out.println("\nBook Name: " + b1.getBookName());  
    }  
}
```

## ■ Compilation & Execution Commands

```
Save as: ConstructionsAndEncasoulation.java  
javac ConstructionsAndEncasoulation.java  
java ConstructionsAndEncasoulation.java
```

## ■■ Output Screen

```
Book 1:  
Book ID: 0  
Book Name: Unknown  
Author: Unknown  
Price: 0.0
```

```
Book 2:  
Book ID: 101  
Book Name: Java  
Author: James Gosling  
Price: 500.0
```

```
After using Setter:  
Book ID: 102  
Book Name: C Programming  
Author: Dennis Ritchie  
Price: 400.0
```

```
Book Name: C Programming
```

## ■ Result

Thus, the Java program demonstrating constructors and encapsulation using getters and setters was successfully implemented and executed.

# EXPERIMENT 04 — IMPLEMENTATION OF HIERARCHICAL INHERITANCE

|               |                                            |
|---------------|--------------------------------------------|
| Experiment No | 04                                         |
| Date          | 17/09/2026                                 |
| Title         | Implementation of Hierarchical Inheritance |

## ■ Objective / Aim

To write a Java program to demonstrate hierarchical inheritance, where two child classes inherit properties from a common parent class.

## ■ System Requirements

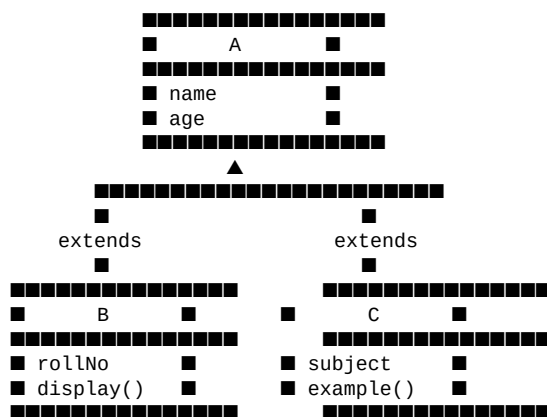
**Hardware:** Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

**Software:** Windows/Linux/macOS, JDK 17/JDK 21, Java IDE/Text Editor.

## ■ Algorithm / Logic Description

1. Create a parent class A.
2. Declare name and age in class A.
3. Create class B that extends A and add rollNo.
4. Create display() in B.
5. Create class C that extends A and add subject.
6. Create example() in C.
7. Create objects of B and C.
8. Call their methods and display the inherited and child class data.

## ■ UML Class Diagram



## ■ Source Code

```
class A {
    String name = "Sai";
    int age = 18;
}

class B extends A {
    int rollNo = 101;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Roll No: " + rollNo);
    }
}
```

```

class C extends A {
    String subject = "Java";

    void example() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Subject: " + subject);
    }
}

public class Inheritance {
    public static void main(String[] args) {
        B s = new B();
        C x = new C();

        s.display();

        System.out.println(" ");

        x.example();
    }
}

```

## ■ Compilation & Execution Commands

```

Save as: Inheritance.java
javac Inheritance.java
java Inheritance.java

```

## ■■ Output Screen

```

Name: Sai
Age: 18
Roll No: 101

```

```

Name: Sai
Age: 18
Subject: Java

```

## ■ Result

Thus, the Java program demonstrating hierarchical inheritance was successfully implemented and executed.

## EXPERIMENT 05 — IMPLEMENTATION OF METHOD OVERLOADING

|               |                                      |
|---------------|--------------------------------------|
| Experiment No | 05                                   |
| Date          | 17/09/2026                           |
| Title         | Implementation of Method Overloading |

### ■ Objective / Aim

To write a Java program to demonstrate method overloading by defining multiple methods with the same name but different parameters.

### ■ System Requirements

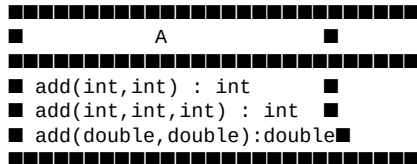
**Hardware:** Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

**Software:** Windows/Linux/macOS, JDK 17/JDK 21, Java IDE/Text Editor.

### ■ Algorithm / Logic Description

1. Create a class named A.
2. Define add() with two integer parameters.
3. Define add() with three integer parameters.
4. Define add() with two double parameters.
5. Create an object of class A.
6. Call each overloaded method with suitable arguments.
7. Display the results.

### ■ UML Class Diagram



### ■ Source Code

```
class A {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Overloading {
    public static void main(String[] args) {
        A a = new A();

        System.out.println("sum is : " + a.add(10, 20));

        System.out.println("sum is : " + a.add(10, 20, 30));

        System.out.println("sum is : " + a.add(30.5, 36.5));
    }
}
```

### ■ Compilation & Execution Commands

```
Save as: Overloading.java  
javac Overloading.java  
java Overloading.java
```

## ■■ Output Screen

```
sum is : 30  
sum is : 60  
sum is : 67.0
```

## ■ Result

Thus, the Java program demonstrating method overloading was successfully implemented and executed.

## EXPERIMENT 06 — IMPLEMENTATION OF METHOD OVERRIDING

|               |                                     |
|---------------|-------------------------------------|
| Experiment No | 06                                  |
| Date          | 17/09/2026                          |
| Title         | Implementation of Method Overriding |

### ■ Objective / Aim

To write a Java program to demonstrate method overriding, where a child class provides its own implementation of a method defined in the parent class.

### ■ System Requirements

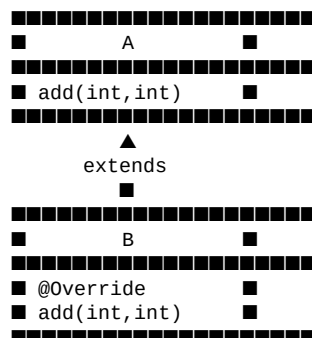
**Hardware:** Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

**Software:** Windows/Linux/macOS, JDK 17/JDK 21, Java IDE/Text Editor.

### ■ Algorithm / Logic Description

1. Create a parent class A.
2. Define an add() method in class A.
3. Create a child class B that extends A.
4. Override the add() method in B.
5. Use the @Override annotation.
6. Create an object of class B.
7. Call add() and display the child class implementation.

### ■ UML Class Diagram



### ■ Source Code

```
class A {
    void add(int a, int b) {
        System.out.println("Parent sum: " + (a + b));
    }
}

class B extends A {
    @Override
    void add(int a, int b) {
        System.out.println("Child sum: " + (a + b));
    }
}

public class Overriding {
    public static void main(String[] args) {
        B b = new B();
        b.add(10, 20);
    }
}
```

```
}
```

## ■ Compilation & Execution Commands

```
Save as: Overriding.java  
javac Overriding.java  
java Overriding.java
```

## ■■ Output Screen

```
Child sum: 30
```

## ■ Result

Thus, the Java program demonstrating method overriding using inheritance was successfully implemented and executed.